

# Software Testing



**Prof. Pradip K. Das**

Professor – Computer Sc. & Engineering Dept.

Head – Centre for Drone Technology

IIT Guwahati, Assam

e-mail: [pkdas@iitg.ac.in](mailto:pkdas@iitg.ac.in)

# Outline

- Defining Software Testing Principles
- What are the types of Software Tests
- What is Test Planning
- Test Execution and Reporting
- Real-Time Testing
- Types of testing
  - Manual testing
  - Automation testing
- Component testing
  - Unit testing
  - Integration testing
- System testing
  - Function testing
  - Structure testing
  - Performance testing
  - Acceptance testing
  - Installation testing

# Defining Software Testing Principles

Objective: To remove as many defects as possible before test since the quality improvement potential of testing is limited.

How to define?

1. **Testing**

The execution of a program to find its faults

1. **Verification**

The process of proving the programs correctness.

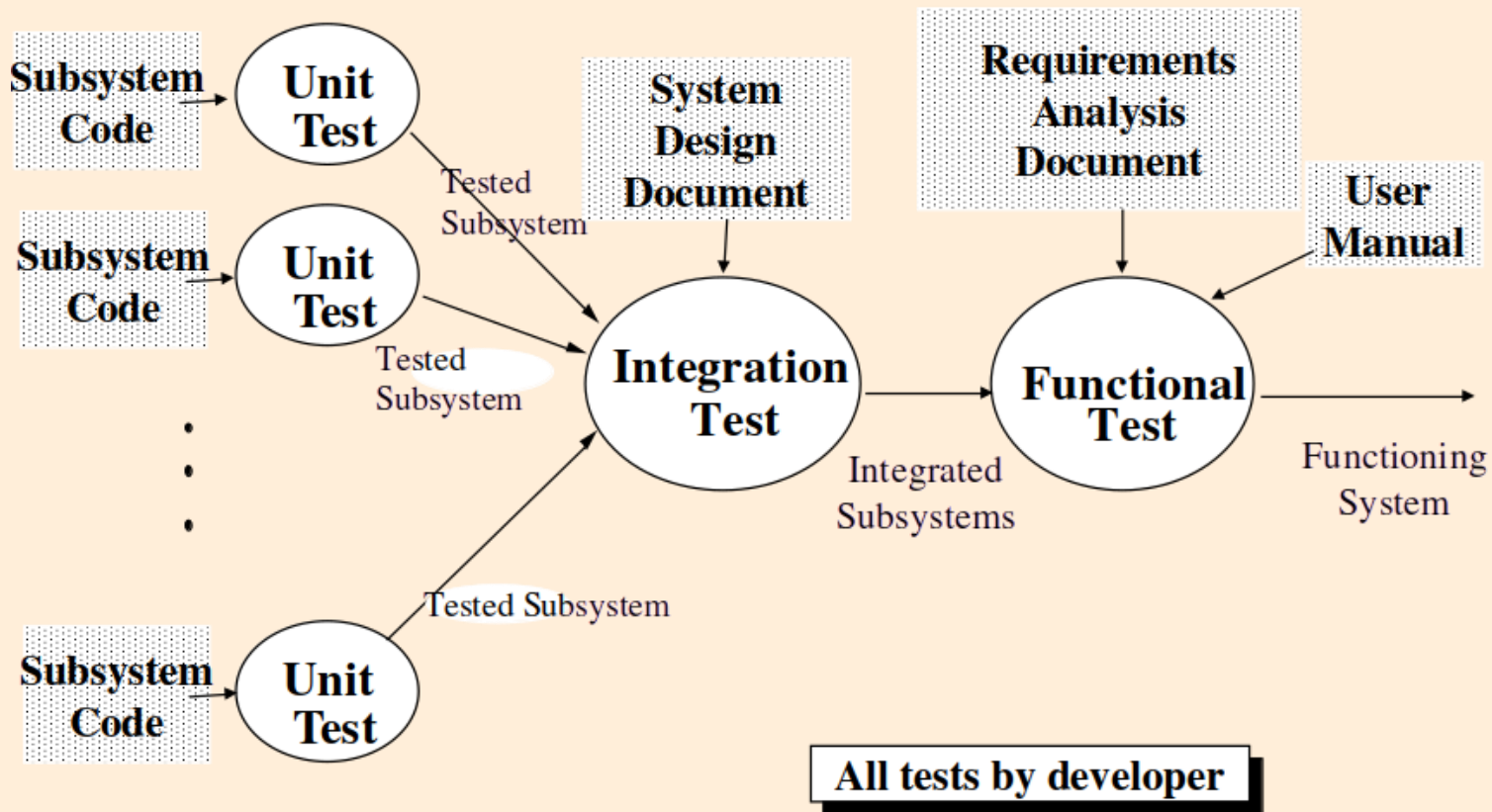
1. **Validation**

The process of finding errors by executing the program in a real environment

1. **Debugging**

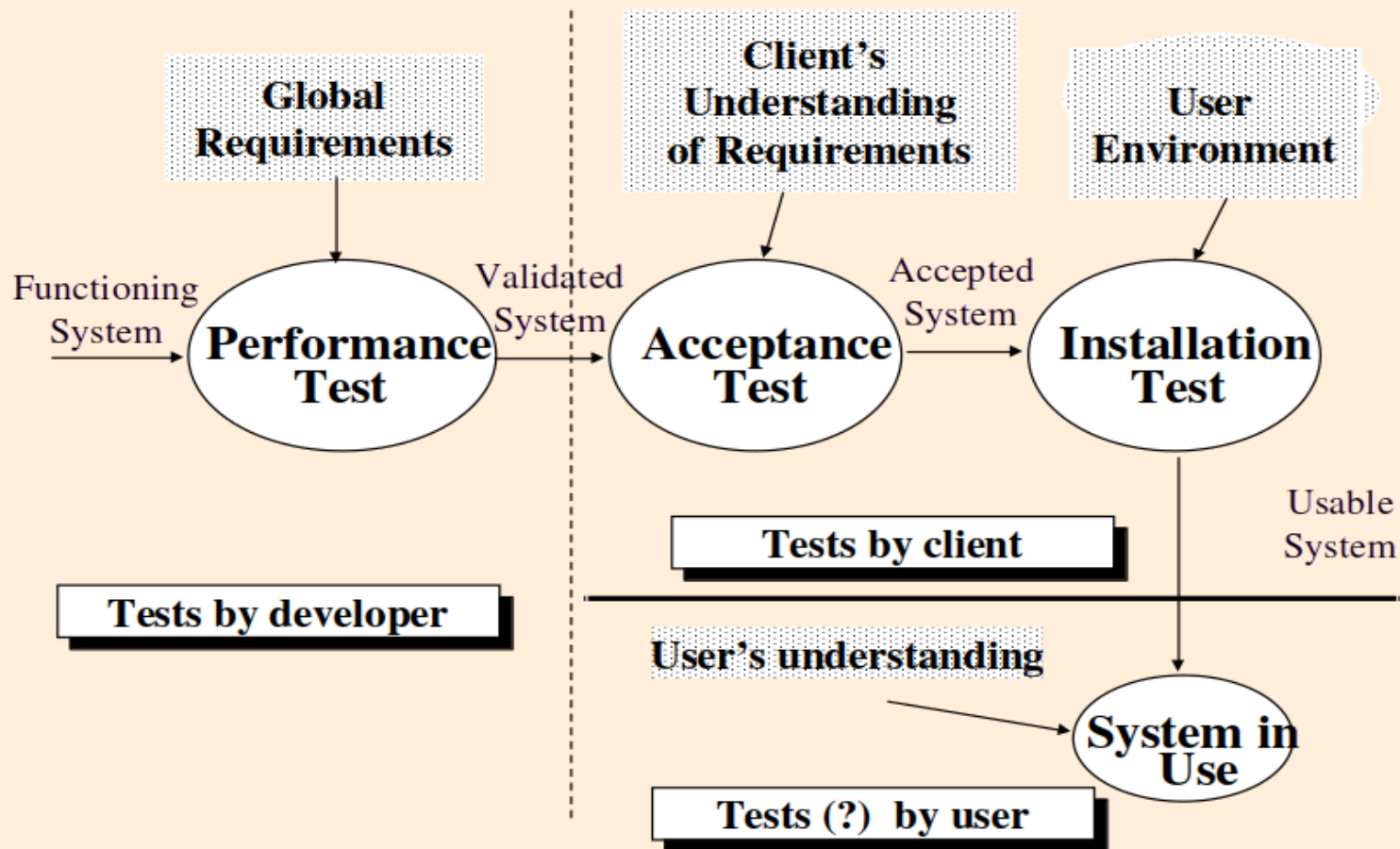
Diagnosing the error and correct it

# Testing Activities

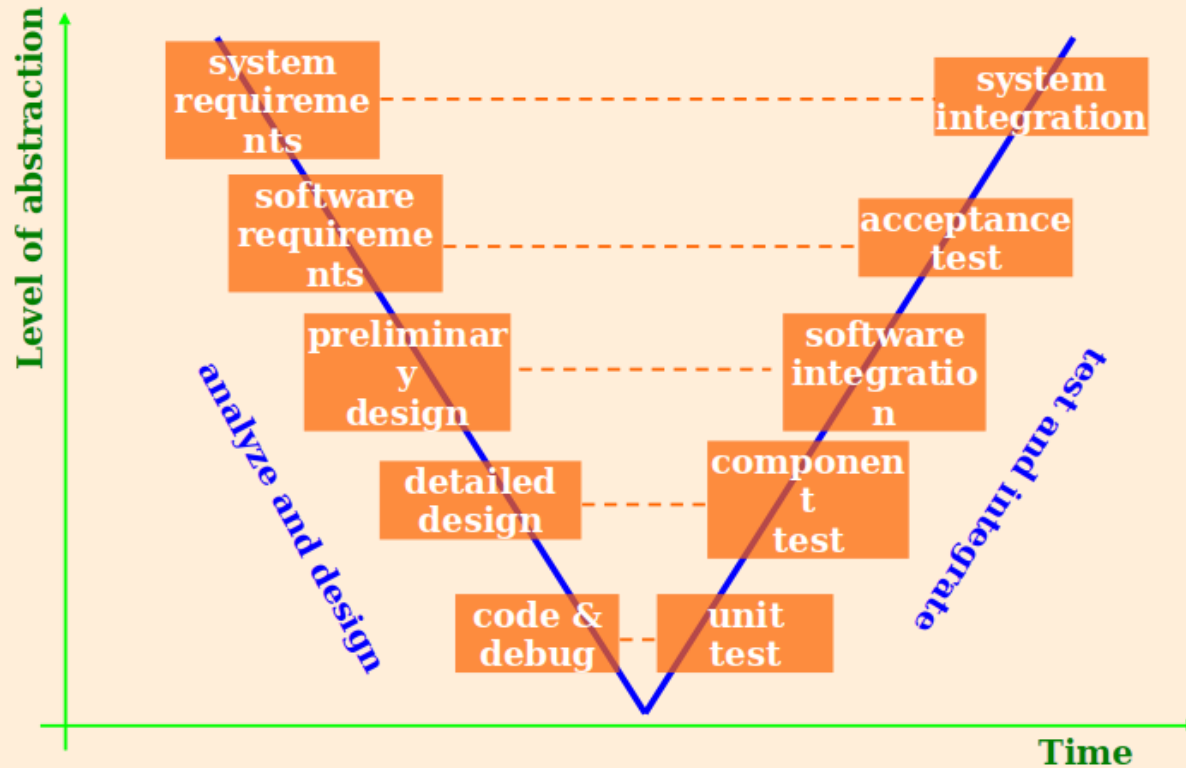


Cf. levels of testing

# Testing Activities continued



# Levels of Testing in V Model



**N.B.: component test vs. unit test; acceptance test vs. system integration**

# *Test Planning*

[Pressman]

- ◆ A Test Plan:
  - ◆ covers all types and phases of testing
  - ◆ guides the entire testing process
  - ◆ who, why, when, what
  - ◆ developed as requirements, functional specification, and high-level design are developed
  - ◆ should be done before implementation starts
- ◆ A test plan includes:
  - ◆ test objectives
  - ◆ schedule and logistics
  - ◆ test strategies
  - ◆ test cases
    - ◆ procedure
    - ◆ data
    - ◆ expected result
  - ◆ procedures for handling problems

## What are the Types of Software Tests

- Unit Testing (White Box)
- Integration Testing
- Function Testing (Black Box)
- Regression Testing
- System Testing
- Acceptance and Installation Tests



# Testing - methods

Black-box testing

White-box testing

Grey-box testing

# Testing - Levels

The main levels of software testing are:

- Functional testing
- Non-functional testing

# Functional testing

Different types of functional testing:

Unit testing

Integration testing

System testing

Regression testing

Acceptance testing

Alpha testing

Beta testing

# Non-functional testing

Types of non-functional testing are:

Performance testing

Load testing

Stress testing

Usability testing

## Unit Testing (White Box)

- Objective: Confirm that subsystems is correctly coded and carries out the intended functionality
- Individual components are tested.
- It is a path test.
- To focus on a relatively small segment of code and aim to exercise a high percentage of the internal path
- **Disadvantage:** the tester may be biased by previous experience. And the test value may not cover all possible values.
  - **Advantage:**
  - **Use cases:**

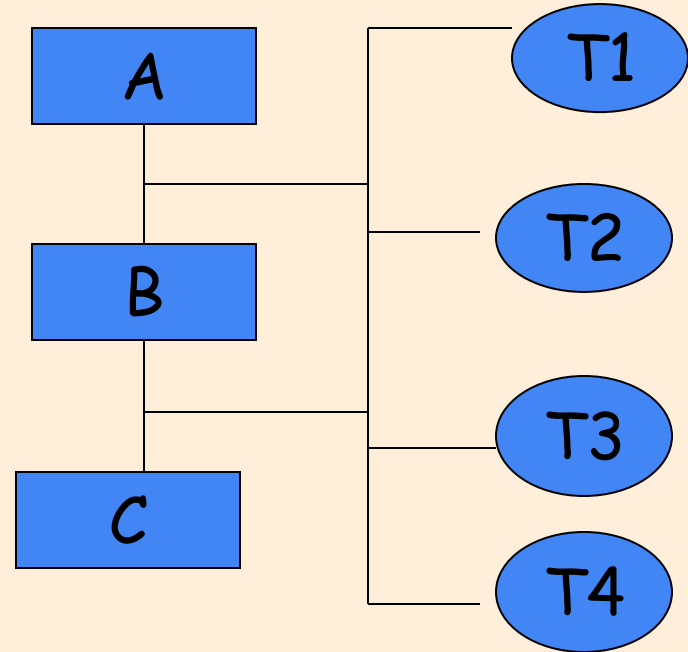
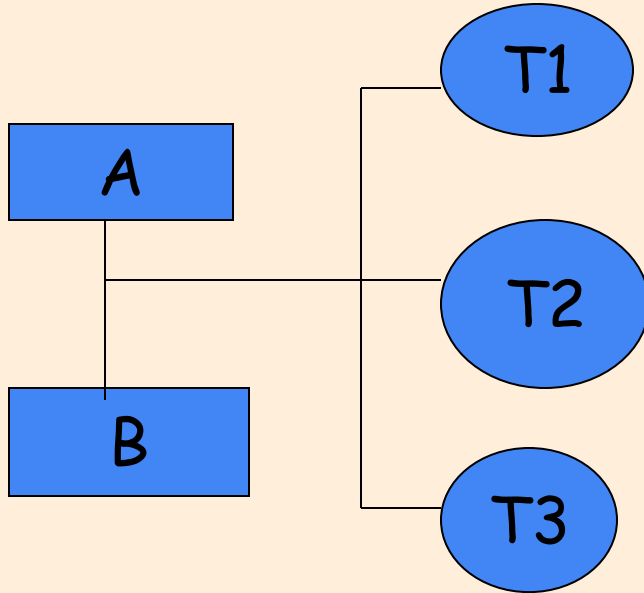
## Integration Testing

- Objective: Test the interface among the subsystem
- Groups of subsystems (collection of classes) and eventually the entire system is tested to verify that all components can work together
- Carried out by developers
- There are two ways to carry out integration testing i.e. **top-down** and **bottom-up**
- Disadvantage:
- Advantage:
- Use cases:

## Top-down Integration Test

- The control program is tested first. Modules are integrated one at a time. Emphasize on interface testing
- Advantages:
  - No test drivers needed
  - Interface errors are discovered early
  - Modular features aid debugging
- Disadvantages:
  - Test stubs are needed
  - Errors in critical modules at low levels are found late.

# Top-down Integration testing

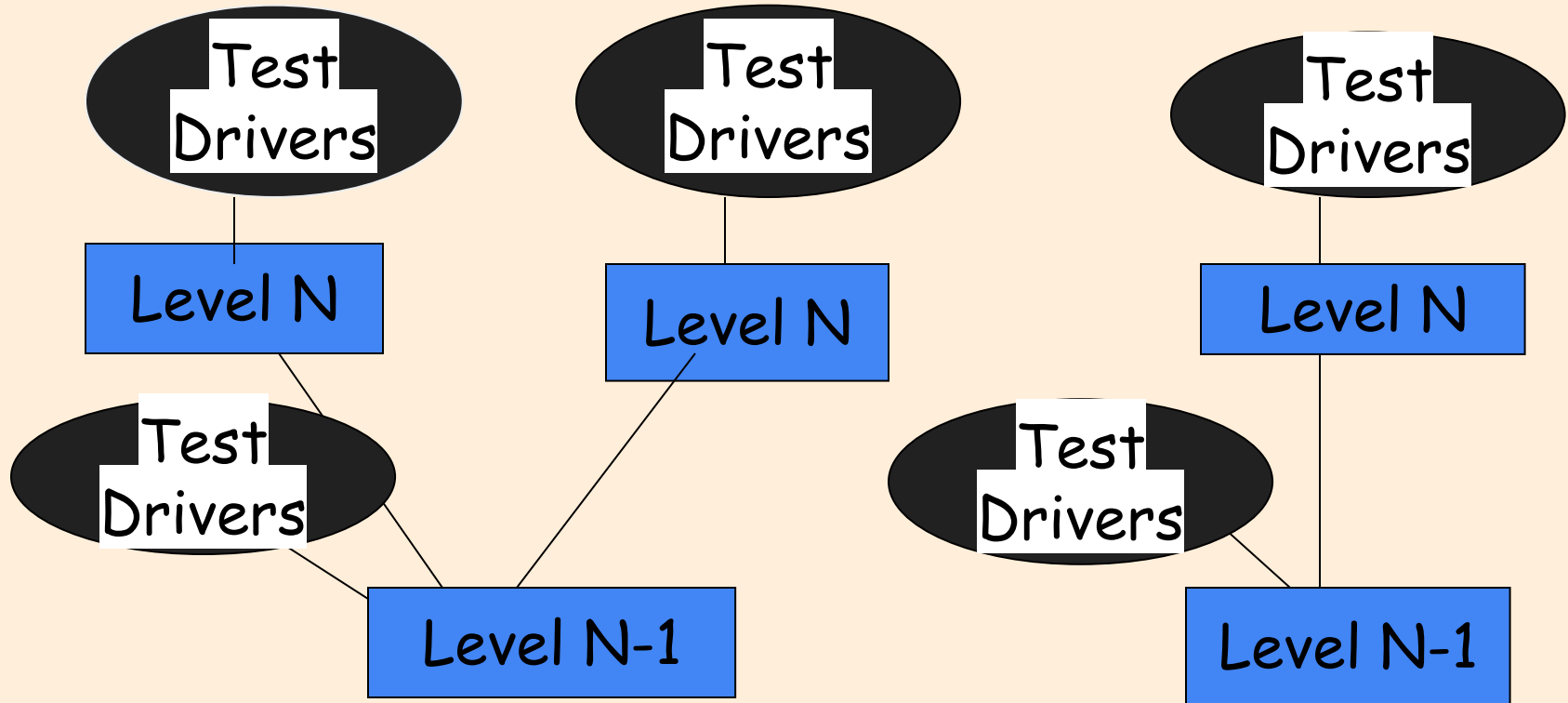




## Bottom-up Integration Test

- Allow early testing aimed at proving feasibility
- Emphasize on module functionality and performance
- Advantages:
  - No test stubs are needed
  - Errors in critical modules are found early
- Disadvantages:
  - Test drivers are needed
  - Interface errors are discovered late

# Bottom-up Integration testing



# Function Testing (Black Box)

- I/O behavior. If for any given input, we can predict the output, then the module passes the test. Almost always impossible to generate all possible inputs ("test cases").
- Goal: Reduce number of test cases by equivalence partitioning:
  - Divide input conditions into equivalence classes.
  - Choose test cases for each equivalence class. (Example: If an object is supposed to accept a negative number, testing one negative number is enough)
- Testers not biased by knowledge of the program's design.
- Disadvantages:
  - The need for explicitly stated requirements
  - Only cover a small portion of the possible test conditions.

## Regression Testing

- Test the effects of the newly introduced changes on all the previously integrated code.
- The common strategy is to accumulate a comprehensive regression bucket but also to define a subset.
- The full bucket is run only occasionally, but the subset is run against every spin.
- Disadvantages:
  - To decide how much of a subset to use and which tests to select.



# System Testing

## ■ System Testing:

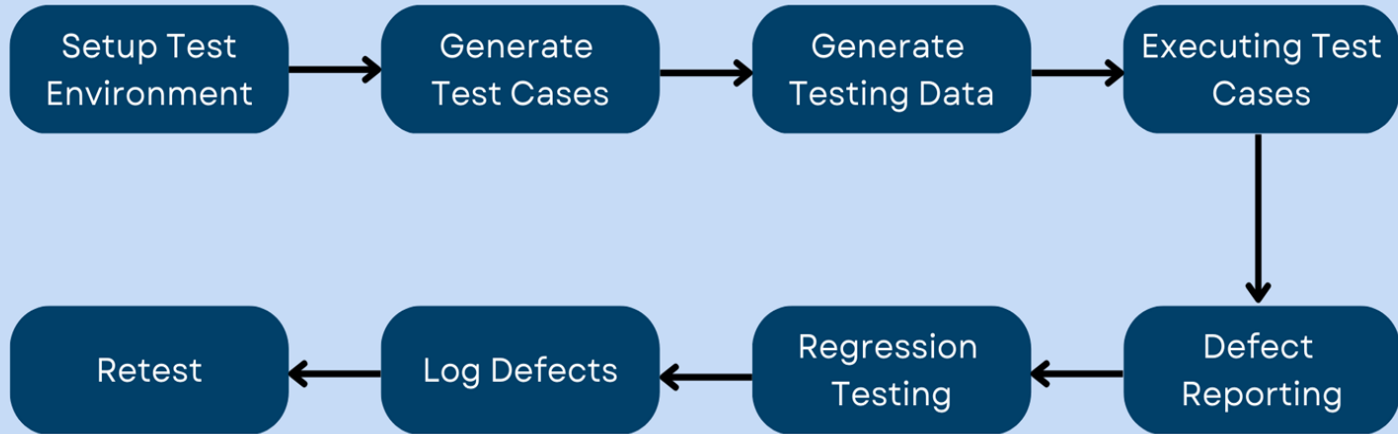
- System testing, also referred to as *system-level testing* or *system integration testing*.
- It is the process in which a quality assurance ([QA](#)) team evaluates how the various components of an application interact together in the full, integrated system or application.
- System testing verifies that an application performs tasks as designed. It's a type of [black box testing](#) that focuses on the functionality of an application rather than the inner workings of a system, which [white box testing](#) is concerned with.
- System testing, for example, might check that every kind of user input produces the intended output across the application. System testing is the third level of testing in the software development process. It's typically performed before [acceptance testing](#) and after [integration testing](#)

- **Goal:** Determine if the system meets the requirements (functional and global)

## Importance of system testing

- System testing is imperative for interconnected systems because any system or software defect can cause extreme complications for the consumers. For example, in 2014, Nissan had to recall over 1 million cars due to a defect in the airbag sensor detectors.

# System Testing :Explained



# Types of system testing





# Acceptance Testing

- **Acceptance Testing:**
  - Evaluates the system delivered by developers
  - Carried out by the **client**. May involve executing typical transactions on site on a trial basis
  - Acceptance Tests are specified by the customer and analyst to test that the overall system is functioning as required (Do developers build the right system?).
  - Acceptance tests typically test the entire system, or some large chunk of it.
  - When all the acceptance tests pass for a given user story (or use case, or textual requirement), that story is considered complete.
  - An acceptance test could consist of a script of user interface actions and expected results that a human can run.
  - Ideally acceptance tests should be automated, either using the unit testing framework, or a separate acceptance testing framework.
- **Goal:** Demonstrate that the system meets customer requirements and is ready to use

## Traditional approaches to acceptance testing

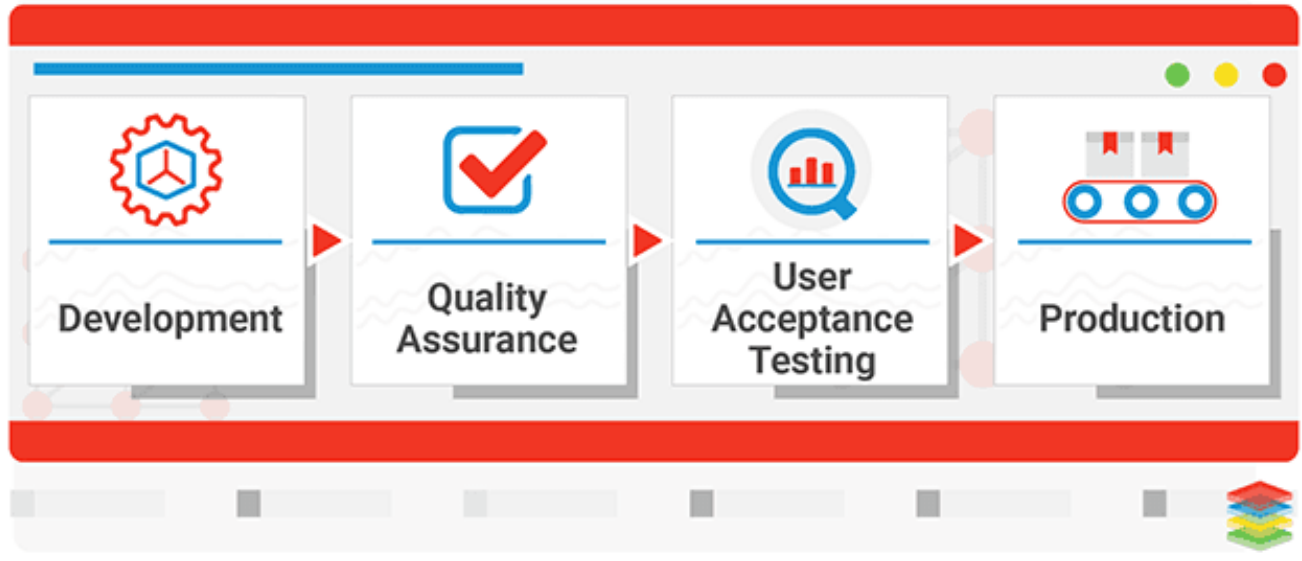
- Manual Acceptance testing. User exercises the system manually using his creativity.
- Acceptance testing with "GUI Test Drivers" (at the GUI level). These tools help the developer to do functional/acceptance testing through a user interface such as a native GUI or web interface. "Capture and Replay" Tools capture events (e.g. mouse, keyboard) in modifiable script.

**Disadvantages:**  
expensive, error prone,  
not repeatable, ...

**Disadvantages:**  
Tests are brittle, i.e.,  
have to be re-captured if  
the GUI changes.

# Acceptance Testing: Explained

## — User Acceptance Testing —



## What is Test Planning?

- Define the functions, roles and methods for all test phases.
- Test planning usually start during the requirements phase.
- Major test plan elements are:
  - Objectives for each test phase
  - Schedules and responsibilities for each test activity
  - Availability of tools, facilities and test libraries
  - Set the criteria for test completion

# Test Execution & Reporting

- Testing should be treated like an experiment.
- Testing require that all anomalous behavior be noted and investigated.
- Big companies keep a special library with all copies of test reports, incident forms and test plans.

# Real-Time Testing

- Real-Time testing is necessary because the deployment system is usually more complicated than development system.
- Rules apply for testing real time system:
  - Evaluate possible deadlocks, thrashing to special timing conditions.
  - Use tests to simulate hardware faults.
  - Use hardware simulation to stress the software design.
  - Design ways to simulate modules missing in the development system.

# References

- DeMillo, Software Testing and Evaluation Benjamin/Cummings Publishing Company, Inc California, 1987.
- Sommerville, Software Engineering Addison-Wesley Publishing Company, 1996
- Humphrey, Managing the Software Process, Addison-Wesley Publishing Company, 1990.